Threaded BST Insertion

Insertion into an Inorder-Threaded Binary Search Tree

Data Structures Project Report • C++

1. Problem Statement

Threaded BST Insertion: Write a C++ program to insert a new node into an inorder-threaded BST and update the affected predecessor and successor threads.

2. Objectives

- **Eliminate Traversal Overhead:** Avoid the need for runtime stacks or recursion during binary search tree (BST) traversal by leveraging thread connections.
- **Efficient Memory Utilization:** Repurpose unused null child pointers (leaves) as directional traversal threads that reference immediate inorder ancestors.
- **Maintain Structural Integrity:** Dynamically link structural parent-child nodes while adjusting neighboring predecessor and successor boundaries upon every insertion.
- **Achieve Linear Traversal Complexity:** Implement a deterministic O(n) iterative sorting loop that evaluates node strings strictly via thread addresses.

3. Algorithm

1. Node Allocation: Allocate memory for node `tmp`. Assign `tmp->data = key`, `tmp->left = nullptr`, `tmp->right = nullptr`, and set both `lthread` and `rthread` flags to true.

2. Base Condition: If `root` equals `nullptr`, point `root` to `tmp`. Print root setup metrics, run `verifyThreads(tmp)`, and return `root`.

3. Traverse Matrix: Set crawler index `ptr = root` and companion pointer `parent = nullptr`.

4. Position Match Loop: Execute while `ptr != nullptr`:

- Set `parent = ptr`.
- If `key == ptr->data`, notify user of duplicate drop, clean up `tmp` memory, and return `root`.
- If `key < ptr->data`: If `ptr->lthread == false`, step left (`ptr = ptr->left`). Else, exit the loop.
- If `key > ptr->data`: If `ptr->rthread == false`, step right (`ptr = ptr->right`). Else, exit the loop.

5. Left Border Placement: If `key < parent->data`:

- Point `tmp->left` to `parent->left` (inherits predecessor context).
- Point `tmp->right` to `parent` (sets parent as its successor target).
- Toggle `parent->lthread = false` (parent left link converts into an explicit child node pointer).
- Assign `parent->left = tmp`.
- Print structural adjustments text block to screen.

6. Right Border Placement: If `key > parent->data`:

- Point `tmp->left` to `parent` (sets parent as its predecessor target).
- Point `tmp->right` to `parent->right` (inherits successor context).
- Toggle `parent->rthread = false` (parent right link converts into an explicit child node pointer).
- Assign `parent->right = tmp`.
- Print structural adjustments text block to screen.

7. Automated Validation Invocations:

- Invoke `verifyThreads(tmp, "Newly Inserted Node Validation")`.
- Invoke `verifyThreads(parent, "Parent Node Link Validation")`.

8. Exit Routine: Return the updated root memory address.

4. Code

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node *left, *right;
    bool lthread;
    bool rthread;
};
Node* insert(Node* root, int key) {
    Node* tmp = new Node();
    tmp->data = key;
    tmp->left = nullptr;
    tmp->right = nullptr;
    tmp->lthread = true;
    tmp->rthread = true;

    if (root == nullptr) {
        root = tmp;
        return root;
    }

    Node* ptr = root;
    Node* parent = nullptr;

    while (ptr != nullptr) {
        if (key == ptr->data) {
            cout << "Duplicate key " << key << " ignored.\n";
            delete tmp;
            return root;
        }

        parent = ptr;

        if (key < ptr->data) {
            if (!ptr->lthread) {
                ptr = ptr->left;
            } else {
                break;
            }
        } else {
            if (!ptr->rthread) {
                ptr = ptr->right;
            } else {
                break;
            }
        }
    }
    if (key < parent->data) {

        tmp->left = parent->left;
        tmp->right = parent;
```

```
        parent->lthread = false;
        parent->left = tmp;
    } else {

        tmp->left = parent;
        tmp->right = parent->right;
        parent->rthread = false;
        parent->right = tmp;
    }

    return root;
}

Node* inorderSuccessor(Node* ptr) {
    if (ptr->rthread) {
        return ptr->right;
    }
    ptr = ptr->right;
    while (!ptr->lthread) {
        ptr = ptr->left;
    }
    return ptr;
}

void printInorder(Node* root) {
    if (root == nullptr) {
        cout << "Tree is empty.\n";
        return;
    }
    Node* ptr = root;
    while (!ptr->lthread) {
        ptr = ptr->left;
    }

    while (ptr != nullptr) {
        cout << ptr->data << " ";
        ptr = inorderSuccessor(ptr);
    }
    cout << "\n";
}

int main() {
    Node* root = nullptr;

    root = insert(root, 20);
    root = insert(root, 10);
    root = insert(root, 30);
    root = insert(root, 5);
    root = insert(root, 15);
    root = insert(root, 25);
    root = insert(root, 35);

    cout << "Inorder Traversal of the Threaded BST: ";
    printInorder(root);

    return 0;
}
```

5. Output

